

Unknot recognition: executable reference code and an audit of the quasi-polynomial hierarchy route

Implementation and technical report

17 September 2026

Delivery status: partial. The requested $n^{O(\log n)}$ implementation has not been completed. The supplied complete recognizer has a proved exponential upper bound and explicitly visits 2^n states. Polynomial-time boundary-pattern code and a hierarchy potential checker are also supplied. They do not constitute a complete implementation of Lackenby’s final flowchart.

Abstract

We examine the computational obligations in the supplied 109-page February 2021 talk on quasi-polynomial unknot recognition. We distinguish its announced bound from the complexity proved for the software in this archive. The executable reference algorithm computes reduced Khovanov homology over $\mathbb{F}_2$ and recognizes the unknot by its total rank. We specify the input representation, construct all differential maps, justify the rank calculation, and prove a $2^{O(n)}$ upper bound. The 2^n resolution loop also proves that this particular implementation is not quasi-polynomial. Separately, we implement the terminal essential-boundary-pattern test under an explicit 3-ball hypothesis and the mixed-radix arithmetic underlying bounded-depth hierarchy simplification. Reproducible tests include braid identities, basepoint independence, independently sourced planar diagrams, a determinant-one nontrivial knot, exact differential-square checks, and resource-limit behavior. The missing geometric algorithms and their required complexity invariants are identified without replacing them by hidden oracles or unproved runtime assertions.

1 What was requested and what is delivered

The input-size parameter n is the number of crossings in the *input diagram*, not the minimum crossing number of its knot type. A genuine solution to the requested task must halt on every classical knot diagram with the correct answer within $n^{O(\log n)}$ time. An algorithm that is usually fast, an exponential algorithm with a time cap, or a framework whose core operations are unspecified does not meet this requirement.

The archive has three executable mathematical components:

Module	Functionality	Actual status
<code>khovanov.py</code>	Complete exact recognition via reduced homology	Exponential, not quasi-polynomial
<code>pattern.py</code>	Essentiality of a pattern on an already known ball	Polynomial graph algorithm
<code>potential.py</code>	Bounded-digit potential and tail-reset audit	Arithmetic only

The supporting modules validate planar diagrams, convert closed braids, and perform exact binary linear algebra. The code is self-contained Python 3.10+ with no third-party runtime or test dependencies. The recorded validation was performed with Python 3.13.5. None of the missing hierarchy operations is presented as an implemented placeholder.

2 The supplied hierarchy route

2.1 Source-derived structure

The supplied talk announces the quasi-polynomial bound on pages 11–14 [1]. Its method uses hierarchies, boundary patterns, and violating discs. The basic flowchart is complete on page 53. It alternates construction of decomposing surfaces with simplification of an earlier surface when a violating disc is found, discarding the appropriate later portion of the hierarchy.

Pages 54–65 discuss lexicographic progress. Pages 66–74 identify four speedups: quadratic surface complexity, compressed hierarchy encodings, multi-surfaces, and Heegaard splittings with Cheeger regions. The last two are intended to force logarithmic hierarchy length. Pages 75–78 emphasize keeping track of the ambient embedding in S^3 when constructing later surfaces. Pages 82–90 distinguish compression of a multi-surface from boundary compression. Pages 91–108 describe the logarithmic-depth strategy and its Cheeger-region alternative.

The final-page flowchart was inspected as an image, including the restart and tail-discard arrows. Its central operations include BUILD HIERARCHICAL MULTI-SURFACE, CUT ALONG SURFACE, USE CHEEGER REGION, SIMPLIFY MULTI-SURFACE, HAKEN’S LEMMA, and WEAKLY REDUCE. These names describe substantial geometric transformations, not interchangeable abstract callbacks. In particular, the Cheeger inequality is not itself the transformation that changes a Heegaard splitting. The slides also explicitly warn that the displayed “genus” may stand for a pattern-sensitive complexity or a version of χ_- . The arithmetic code therefore accepts nonnegative complexity digits; it does not mistake ordinary genus for the invariant needed by every geometric step.

2.2 Outside research and its limits

Lackenby’s July 2026 preprint [2], Section 9, distinguishes the number of hierarchy iterations from the running time of individual steps. It states that some steps have not been specified precisely enough for a running-time estimate, and gives an iteration bound $L(g+1)^L$. Its Section 10 obtains a linear bound on hierarchy length in the initial q-complexity. These statements do not by themselves specify the logarithmic-depth implementation announced in the slides. We do not infer from this that the announcement is false, or that no other manuscript or implementation exists.

Proposition 2.6 of that preprint supplies a finite graph criterion for an essential pattern on a ball.

That is the additional source used for `pattern.py`. The reference recognizer follows a different method: Khovanov homology, already listed among alternatives on page 8 of the supplied talk. It is explicitly not a port of the final hierarchy flowchart.

2.3 An exact arithmetic consequence, with explicit hypotheses

Proposition 2.1 (Bounded hierarchy potential). *Fix integers $g, L \geq 0$. For a complexity list $\gamma = (\gamma_1, \dots, \gamma_\ell)$ with $\ell \leq L$ and $0 \leq \gamma_i \leq g$, define*

$$\Phi(\gamma) = \sum_{i=1}^{\ell} \gamma_i (g+1)^{L-i}.$$

Suppose a simplification decreases γ_j by at least one, preserves the preceding entries, and replaces the following entries by any list satisfying the same bounds. Then Φ decreases by at least one.

Proof. Write $B = g + 1$. The decrease at position j contributes at least B^{L-j} . The largest possible increase of the entire tail is

$$g \sum_{k=0}^{L-j-1} B^k = B^{L-j} - 1.$$

Their difference is at least one. When $g = 0$, no strictly decreasing entry is available, so the implication is vacuous. The padded list has a value between zero and $B^L - 1$. $\square$

The code checks this operation exactly, using integer arithmetic. The choice of base $g + 1$ is intentional for the inclusive digit bound $0, \dots, g$. With base g , for example, the lists $(1, 0)$ and $(0, g)$ have equal value. This is a bookkeeping clarification, not a refutation of the announced complexity bound.

One must compare the potential at the appropriate construction-loop exits. Individual cuts during reconstruction may increase it. At most L construction steps between simplifications gives the familiar factor $L(g+1)^L$; handling initialization or $L = 0$ adds a constant term.

Corollary 2.2 (Conditional quasi-polynomial accounting). *Suppose a fully implemented hierarchy algorithm has, uniformly on all n -crossing inputs, a polynomial digit bound $g \leq C(n+2)^a$, a depth bound $L \leq C' \log(n+2)$, polynomial bit-cost per counted step, and only polynomially many outer restarts. If its operations satisfy the progress hypothesis above, then its running time is $(n+2)^{O(\log(n+2))}$.*

Proof. The logarithm of $(g+1)^L$ is $L \log(g+1) = O((\log(n+2))^2)$. Polynomial factors and the factor $L+1$ do not change this bound. $\square$

These are sufficient hypotheses, not properties established for any geometric implementation in this archive. In particular, `potential.py` cannot prove that a list of integers comes from surfaces, that their complexities satisfy a quadratic bound, or that a Cheeger restart decreases an appropriate outer potential.

3 A complete reference recognizer

3.1 Planar-diagram representation and validation

A crossing is a cyclic tuple (a, b, c, d) in counterclockwise order, starting at an underpassing port. Opposite ports form each strand. Each arc label appears exactly twice. Labels are nonnegative integers and are normalized on input. A marked arc specifies the basepoint used in reduction. The empty PD is defined to represent one crossing-free circle; it does not represent an empty link or an unspecified number of crossing-free components.

Let α pair ports carrying the same arc label and let σ rotate the four ports at every crossing. A disjoint-set computation using α and opposite-port pairings counts knot components. Input with more than one component is rejected. For a one-component diagram, the underlying projection graph is connected. Its faces are the cycles of $\sigma\alpha$. With n vertices and $2n$ edges, its associated closed orientable ribbon surface has Euler characteristic

$$\chi = n - 2n + \# \text{cycles}(\sigma\alpha).$$

The code requires $\chi = 2$. Thus it checks planarity of the *specified rotation system*, not merely planarity of its underlying abstract graph. Nonplanar or virtual diagrams are rejected before invoking a classical-knot detection theorem.

The braid front end tracks successive strand segments, puts the chosen crossing over/under data into this cyclic convention, and pairs bottom segments with top segments under closure. It first checks that the braid permutation has exactly one cycle. This prevents an untouched strand from silently becoming an omitted crossing-free link component. Positive generators pass the upper-left strand over the upper-right strand. A global mirror convention difference with another package does not change the recognition answer or total rank.

3.2 Resolutions and reduced chain groups

For a state $s \in \{0, 1\}^n$, use the two smoothing rules

$$0 : (a, b), (c, d), \quad 1 : (a, d), (b, c).$$

A disjoint-set structure on arc labels computes the resulting circles. Order them by their first normalized arc. Let $k(s)$ denote the number of circles and $p(s)$ the circle containing the marked arc. Use the Frobenius algebra

$$A = \mathbb{F}_2[X]/(X^2), \quad \{1, X\} \text{ as basis.}$$

The unreduced summand is $A^{\otimes k(s)}$. Its reduced subspace consists of basis tensors whose marked circle is labelled X . Therefore

$$\tilde{C}_s \cong \mathbb{F}_2^{2^{k(s)}-1}, \quad \tilde{C}_h = \bigoplus_{|s|=h} \tilde{C}_s.$$

These are the standard reduced-chain conventions, without the normalization shifts in homological and quantum degrees [3, 4, 5]. Those shifts have no effect on the total rank used for recognition.

A basis vector is encoded by one bit per unmarked circle: zero means 1 and one means X . The implementation inserts the forced X bit at position $p(s)$ before applying a saddle map and removes it afterward. An invariant check rejects a target tensor that leaves this reduced subcomplex.

3.3 All differential maps

An edge of the resolution cube changes one zero bit to one. It either merges two circles or splits one circle into two. On the changed factors the maps are

$$\begin{aligned} m(1 \otimes 1) &= 1, & m(1 \otimes X) &= X, & m(X \otimes 1) &= X, & m(X \otimes X) &= 0, \\ \Delta(1) &= 1 \otimes X + X \otimes 1, & \Delta(X) &= X \otimes X. \end{aligned}$$

The unchanged circles are identified by their arc sets, so no implicit permutation of tensor factors is left unspecified. The function `Saddle.between` identifies the changed circles by intersections of the source and target partitions. It checks that the changed counts are exactly $(2, 1)$ or $(1, 2)$.

The coefficient of an output tensor is recorded by toggling its bit in the target column. Consequently coincident contributions cancel over $\mathbb{F}_2$. The total differential $d_h : \tilde{C}_h \rightarrow \tilde{C}_{h+1}$ is the sum over zero-to-one cube edges. Cube signs can be omitted in characteristic two. Associativity, coassociativity, and the Frobenius identity make each square commute; its two length-two paths therefore cancel, giving $d_{h+1}d_h = 0$. If the marked circle participates in a saddle, the displayed tables preserve its X label or give zero. Thus the construction restricts to the reduced complex.

The optional `-verify-d2` mode also checks $d_{h+1}d_h = 0$ by exact matrix composition on every basis vector. This detects a useful class of implementation errors; it does not replace the mathematical invariance theorem or constitute a formal proof of the Python program.

3.4 Exact ranks and the recognition theorem

Let $c_h = \dim_{\mathbb{F}_2} \tilde{C}_h$ and $r_h = \text{rank}_{\mathbb{F}_2} d_h$, taking nonexistent boundary ranks to be zero. Then

$$b_h = c_h - r_{h-1} - r_h, \quad \sum_h b_h = \sum_h c_h - 2 \sum_h r_h. \quad (1)$$

The code computes each r_h by column elimination over $\mathbb{F}_2$. A column is a nonnegative Python integer. Its highest set bit determines its pivot. XOR with an existing pivot clears that leading position. A nonzero column with an unused leading position is stored as a new pivot. Distinct leading positions prove independence, while each eliminated column is in the span of the stored columns. Thus the number of pivots is exactly the matrix rank. No floating-point approximation or random choice occurs.

Kronheimer and Mrowka prove that a nontrivial classical knot has reduced integral Khovanov homology of rank greater than one [6]. This is the deep theorem used by the recognizer, not reproved here.

Theorem 3.1 (Correctness of the reference decision rule). *For a classical knot K ,*

$$K \text{ is the unknot} \iff \dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2) = 1.$$

Consequently the implemented resolution construction, exact elimination, and rule (1) return the correct unknot verdict whenever the computation completes without a resource interruption or implementation failure.

Proof. The reduced complex is the reduction modulo two of a finite complex of free abelian groups. The universal coefficient theorem gives

$$\dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2) \geq \text{rank}_{\mathbb{Z}} \widetilde{\text{Kh}}(K; \mathbb{Z}).$$

For a nontrivial knot the right side is greater than one by the cited theorem. For the crossing-free unknot there is one marked circle and exactly one reduced generator, with zero differential, so the left side is one. Khovanov invariance extends this to every unknot diagram. Formula (1) computes precisely the left side. $\square$

A dimension computation of a finite complex always terminates with unlimited resources. All dimensions and matrix entries are finite. The implementation therefore specifies a total reference decision algorithm on valid inputs in the usual unbounded-memory computational model. Resource-limited execution is deliberately a three-result interface: `UNKNOT`, `KNOTTED`, or `UNKNOWN`.

4 The actual complexity of the reference code

Lemma 4.1. *For a connected n -crossing projection and any resolution s , $k(s) \leq n + 1$.*

Proof. Collapse each resolution circle to a vertex and restore a band at every crossing. The resulting graph is connected, because restoring the crossing neighborhoods recovers the connected projection. It has $k(s)$ vertices and n edges, possibly including loops and repeated edges. A connected graph has at least one fewer edge than vertices. $\square$

Set

$$D = \sum_{s \in \{0,1\}^n} 2^{k(s)-1}.$$

The lemma gives $D \leq 2^n 2^n = 4^n$. Resolution partitions require at most $\text{poly}(n)2^n$ work and storage. In a conservative dense-bit analysis, a matrix column has at most D bits. There are at most D columns and at most D pivot eliminations per column. The resulting rank cost is $\text{poly}(n)D^3$ bit operations; column construction and optional d^2 checks also fit a polynomial in n, D . A bound $\text{poly}(n)(D^2 + 2^n)$ suffices for memory. In particular,

$$T(n) = 2^{O(n)}, \quad S(n) = 2^{O(n)}. \quad (2)$$

Arbitrary large external labels add polynomial preprocessing in their input bit length. With ordinary normalized labels this does not alter (2).

There is also a decisive lower bound *for this code*. Its build loop iterates over every integer from zero to $2^n - 1$. Even diagrams of the unknot with many added kinks take this path. Hence its unrestricted worst-case time is at least $\Omega(2^n)$. Since

$$\frac{n}{(\log n)^2} \longrightarrow \infty,$$

this is not bounded by $n^{C \log n}$ for any fixed C . Thus the exponential label is not merely a pessimistic analysis concealing a proved quasi-polynomial implementation.

Bit packing improves constants and the reference is useful for small examples; it does not remove the resolution cube. Neither this calculation nor the test timings proves a lower bound for *other* unknot-recognition algorithms.

5 The terminal boundary-pattern component

The module `pattern.py` works under the explicit promise that the ambient 3-manifold is a 3-ball. A pattern is supplied as cyclic triples of signed edge ends, together with a count of simple closed curves without vertices. Each edge occurs once with each sign. Each connected graph component's rotation system is checked to have spherical Euler characteristic.

The implemented criterion is that the pattern is empty, a single simple closed curve, or a connected graph whose dual is simple and either 4-connected or K_3 or K_4 [2, Proposition 2.6]. Multiple pattern components are inessential. The K_3 exception matters: the theta graph on a sphere has this dual and is essential.

For a connected graph, the code computes the dual by assigning each halfedge to its face and joining the faces on its two sides. It rejects a loop or a repeated dual edge. After the two small complete-graph exceptions, it tests every deletion set of at most three dual vertices and searches the remainder for disconnected components. With m edges in the cubic primal graph, there are $O(m)$ faces, $O(m^3)$ deletion sets, and $O(m)$ work per traversal, giving $O(m^4)$ combinatorial operations. Label processing adds polynomial bit cost.

A negative result carries an explicit *graph obstruction*: a loop edge, a pair of repeated edges, a small vertex separator, or disconnectedness of the original pattern. A positive result states essentiality *on the assumed ball*. Neither result recognizes the ball itself. Nor does a separator construct a normal compression disc, recover its image under a regluing map, or identify the last multi-surface it crosses. Those are separate missing geometric operations from the final flowchart.

The tests include the theta pattern, tetrahedral pattern, cube pattern, triangular prism, bridges, double-edge obstructions, disconnected patterns, and a positive-genus rotation system. The cube has simple 4-connected dual and is accepted. The prism has a dual separating triple and is rejected.

6 Validation and independent fixture data

The reproducible command is

```
python run_validation.py
```

It runs 62 test methods and then computes 12 knot examples and three boundary patterns. The full results are in `validation/unittest.log` and `validation/results.json`; no network access is required.

The tests compare bit-vector elimination against a separate dense binary matrix implementation on 360 matrices. They also check all 168 one-component three-braid words of lengths two and four, random six-crossing words, cancellation pairs inserted at multiple positions, braid relations, commutation of distant generators, conjugation, and positive and negative Markov stabilization. Tests vary the marked arc, crossing order, arc labels, and mirror. Exact differential-square checks are performed on these cases. Malformed, multi-component, and nonplanar inputs are tested as well.

Representative measured algebraic outputs are shown below. These are computed results, not an extrapolation from a performance benchmark.

Input	n	Resolutions	D	Reduced $\mathbb{F}_2$ rank
Crossing-free unknot	0	1	1	1
One-kink unknot	1	2	3	1
Cancellation-pair unknot	3	8	15	1
Trefoil	3	8	15	3
Figure-eight	4	16	33	5
Cinquefoil	5	32	123	5
Square knot	6	64	225	9
$T(3, 4)$ braid	8	256	801	5
$T(3, 5)$ braid	10	1024	4209	7
10_{124} Atlas PD	10	1024	10617	7

The trefoil, figure-eight and 10_{124} PD fixtures were separately obtained from Knot Atlas [7], rather than generated by this implementation’s braid converter. The 10_{124} entry identifies $T(3, 5)$ and has determinant one. Its recognition as nontrivial therefore exercises more than a non-unit-determinant test. The algorithm actually computes homology; no Alexander or Jones polynomial filter supplies the answer.

There is also an independent numerical check from the Atlas integral unreduced Khovanov table for 10_{124} . It has total free rank 10 and two $\mathbb{Z}/2$ summands, hence total unreduced dimension 14 over $\mathbb{F}_2$ by the universal coefficient theorem. The reduced dimension is therefore 7. For completeness, the two-copies relationship needed here has an elementary chain-level explanation.

Let x be multiplication by X at the marked circle and let ν be the sum, over all circles, of the operator replacing X by 1 and killing 1. Over $\mathbb{F}_2$, the displayed merge and split tables show that ν commutes with the differential. Also $x^2 = \nu^2 = 0$ and

$$x\nu + \nu x = 1.$$

The reduced subcomplex is $R = \text{im } x = \ker x$. Every vector v splits as $x\nu v + \nu x v$, and the sum $R + \nu R$ is direct: if $\nu r \in R$ then $0 = x\nu r = r$. Moreover $\nu : R \rightarrow \nu R$ is an isomorphism, inverse to x . Thus the unreduced complex is the direct sum of two copies of the reduced complex, ignoring grading shifts. This justifies the table-to-reduced-rank check without confusing rational and characteristic-two dimensions.

No independent Regina or KnotTheory executable was run, and the Python code was not formally verified in Lean. Source-table comparisons, algebraic identities, and regression tests are evidence about the implementation, not a formal end-to-end proof. They provide no evidence for the unimplemented quasi-polynomial bound.

7 Remaining obligations for the requested algorithm

The source audit in `source_audit.md` maps individual slide ranges to implementation status. The outstanding tasks are concrete.

Encoded states and ambient information. Choose an explicit finite representation of layered handle structures, thin and thick surfaces, boundary patterns, ambient embeddings, and regluing maps. Prove a bound on the bit size of all maintained data relative to the original input. A normal coordinate vector can be short while its represented surface has exponentially many pieces.

Constructive multi-surfaces. Select independent classes in relative homology and construct their embedded representatives with the necessary pattern-complexity bound. Preserve the independence and representation invariants under compression. Existence of a surface of small genus does not bound the cost of searching for it.

Compressed cuts and certificates. Implement the actual cuts, component tracking, orientation, Euler characteristic, boundary patterns, and certificate lifting without expanding all normal discs. Compression algorithms require a precise surrounding data model. The returned graph obstruction must be converted to a violating disc with enough information to trace it through prior cuts.

Simplification and outer progress. Implement the multi-surface simplification and weak-reduction transformations, including the different tail effects of compression and boundary compression. A constructive Cheeger routine must verify its Morse-theoretic hypotheses, produce a modified splitting, and satisfy an outer progress measure. It cannot be replaced by a scalar inequality on genera or an arbitrary restart.

Complete bit-cost proof. Bound the cost of every operation, every intermediate representation, reconstructed tail, and outer restart. Establish the logarithmic depth and polynomial complexity bounds for *these executable choices*. Only then does the conditional accounting theorem apply.

These are precisely the parts not completed in this archive. Substituting normal-surface enumeration, Reidemeister search, or the reference Khovanov engine into one of these boxes would not inherit the announcement’s running time. Likewise, returning UNKNOWN after a quasi-polynomial cutoff would not turn an incomplete search into a total recognition algorithm.

8 Use and operational safeguards

From the archive root, for example:

```
python -m unknot recognize examples/trefoil.json --verify-d2
python -m unknot recognize examples/torus_3_5.json --max-states 1000
python -m unknot pattern examples/pattern_cube.json
```

The first command returns KNOTTED with rank 3. The second returns UNKNOWN, since 10 crossings require 1024 states. The third reports an essential pattern, conditional on the explicitly stated ball hypothesis. Every recognition result carries `quasipolynomial_guarantee: false`.

Unrestricted recognition is the default. Optional state and generator budgets are checked before the relevant growth. The time budget is cooperative and does not preempt individual Python operations; parsing and validation precede the timed region. Python memory-allocation failures and keyboard interrupts produce no topological verdict, but a hard operating-system kill cannot be caught. Invalid input and internal invariant failures have distinct exit statuses. The ordinary output is diagnostic data, not a succinct independently verified isotopy or hierarchy certificate.

Conclusion

The complete executable contribution is an exact exponential reference recognizer. The graph and arithmetic modules implement limited, well-defined parts relevant to the hierarchy route. The requested end-to-end $n^{O(\log n)}$ implementation remains unfinished in this delivery; no statement about its complexity is inferred from unnamed procedures or favorable tests.

References

- [1] M. Lackenby, *Unknot recognition in quasi-polynomial time*, February 2021. User-supplied `quasipolynomial-talk.pdf`, 109 pages. References in this report use the PDF page count of that file.
- [2] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, 25 July 2026, arXiv:2607.23350v1. <https://arxiv.org/abs/2607.23350>.
- [3] M. Khovanov, *A categorification of the Jones polynomial*, arXiv:math/9908171. <https://arxiv.org/abs/math/9908171>.
- [4] D. Bar-Natan, *On Khovanov’s categorification of the Jones polynomial*, Algebraic & Geometric Topology **2** (2002), 337–370. <https://arxiv.org/abs/math/0201043>.
- [5] M. Khovanov and R. Lipshitz, *Categorical lifting of the Jones polynomial: a survey*, arXiv:2202.02473v2. <https://arxiv.org/abs/2202.02473>.
- [6] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, arXiv:1005.4346, especially the abstract and Section 1.1. <https://arxiv.org/abs/1005.4346>.
- [7] Knot Atlas, entries 3_1 , 4_1 , and 10_{124} , planar-diagram presentations and integral Khovanov tables, accessed 17 September 2026. https://katlas.org/wiki/3_1, https://katlas.org/wiki/4_1, https://katlas.org/wiki/10_124. Numerical fixture provenance is also recorded in `fixture_sources.json`.